

COMPTE RENDU

BI & Datawarehouse – Projet 2 : Chargement et Historisation des
Données avec SSIS – Alimentation d'un Entrepôt de Données et Slowly
Changing Dimensions
3e année Cybersécurité – École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
JIDAL Ilyas
KABORE Mohammad Sharif Jonathan

Filière : Cybersécurité

Année : 2025/2026

Enseignants : M.RIADSOLH

Date : 27 avril 2026

Table des matières

Introduction	2
Architecture de l'entrepôt de données	2
1 Partie A — Chargement des données	3
1.1 Chargement via SSMS (requête SQL directe)	3
1.2 Chargement via SSIS	4
1.2.1 SQL Task — SQLTaskLoadCustomers	4
1.2.2 Data Flow — LoadProduct	4
1.2.3 Data Flow — Load Date	5
1.2.4 Data Flow — LoadSales	5
1.3 Ordre d'exécution et Sequence Container	7
1.4 Récapitulatif des données chargées	7
2 Partie B — Slowly Changing Dimensions (SCD)	8
2.1 Contexte et préparation	8
2.2 Configuration du SCD Wizard	8
2.2.1 Mapping des colonnes et types SCD	8
2.2.2 Options historiques	9
2.2.3 Data Flow SCD généré	9
2.3 Exécution et résultats SCD	11
2.4 Historisation des suppressions	11
2.4.1 Architecture du Data Flow	11
Conclusion	13

Introduction

Ce laboratoire a pour objectif de mettre en pratique le chargement de données dans un entrepôt de données (*Data Warehouse*) en utilisant différentes approches offertes par SQL Server Integration Services (SSIS). Dans un second temps, nous implémentons la gestion des *Slowly Changing Dimensions* (SCD) afin d'historiser les modifications et suppressions dans la dimension Customers.

Architecture de l'entrepôt de données

L'entrepôt de données DWH est composé de trois tables de dimensions et d'une table de faits, formant un schéma en étoile (*star schema*) :

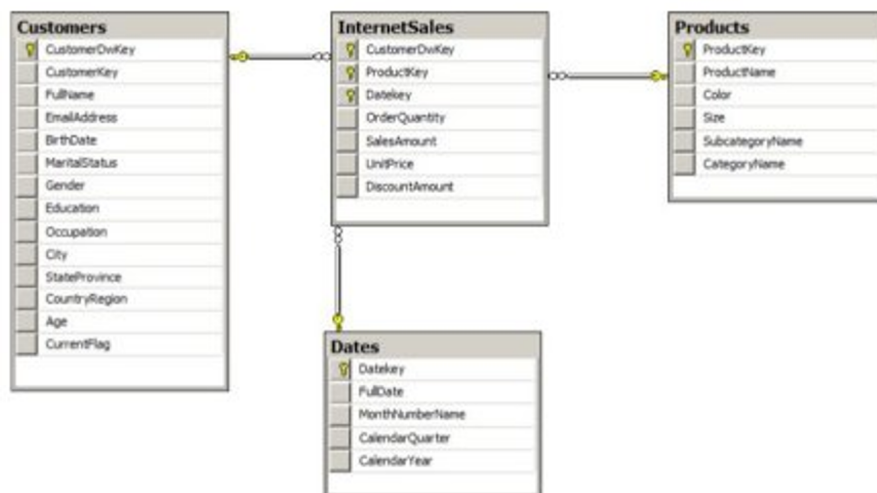


FIGURE 1 – Schéma en étoile de l'entrepôt DWH

- **Customers** : dimension clients avec informations géographiques et démographiques
- **Products** : dimension produits avec catégories et sous-catégories
- **Dates** : dimension temporelle
- **FactSales** : table de faits contenant les ventes Internet

1 Partie A — Chargement des données

1.1 Chargement via SSMS (requête SQL directe)

La première approche consiste à exécuter directement une requête `INSERT INTO ... SELECT` depuis SSMS. Cette méthode extrait les données de la base source `AdventureWorksDW2017` et les insère dans la table `dbo.Customers` de l'entrepôt DWH.

Listing 1 – Chargement de la table Customers depuis SSMS

```
1 USE DWH;  
2 INSERT INTO dbo.Customers  
3 (CustomerKey, FullName, EmailAddress, BirthDate, MaritalStatus,  
4  Gender, Education, Occupation, City, StateProvince,  
5   CountryRegion)  
6 SELECT  
7   C.CustomerAlternateKey,  
8   COALESCE(C.FirstName+' ', '') + COALESCE(C.LastName, '') AS  
9   FullName,  
10  C.EmailAddress, C.BirthDate, C.MaritalStatus, C.Gender,  
11  C.EnglishEducation, C.EnglishOccupation,  
12  G.City, G.StateProvinceName, G.EnglishCountryRegionName  
13 FROM AdventureWorksDW2017.dbo.DimCustomer AS C  
14 INNER JOIN AdventureWorksDW2017.dbo.DimGeography AS G  
15   ON C.GeographyKey = G.GeographyKey;
```

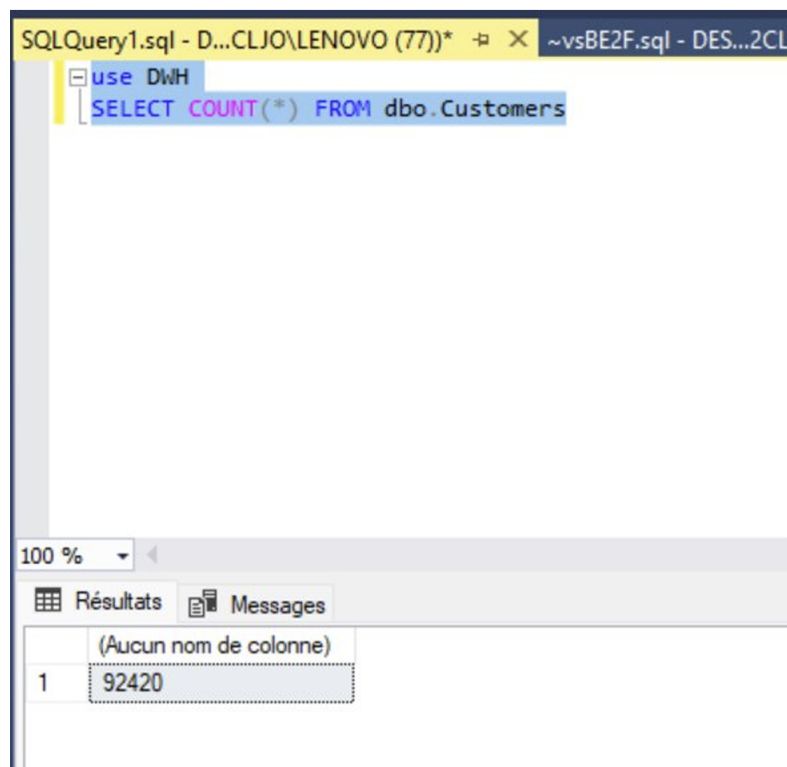


FIGURE 2 – Résultat du chargement — 92 420 lignes dans la table Customers

Réponse

Pourquoi la table **Customers** est-elle considérée comme dénormalisée ?

La table **Customers** est dénormalisée car elle regroupe dans une seule table des données provenant de plusieurs tables sources : **DimCustomer** (informations personnelles) et **DimGeography** (informations géographiques). Dans un modèle normalisé, ces données seraient séparées en plusieurs tables reliées par des clés étrangères. Ici, toutes les informations sont consolidées en une seule table, ce qui élimine les jointures lors des requêtes analytiques et améliore les performances de lecture — principe fondamental des entrepôts de données.

1.2 Chargement via SSIS

Nous avons créé un package SSIS `load_by_SSIIS.dtsx` avec deux gestionnaires de connexions OLE DB :

- **Source** : AdventureWorksDW2017
- **Destination** : DWH

1.2.1 SQL Task — **SQLTaskLoadCustomers**

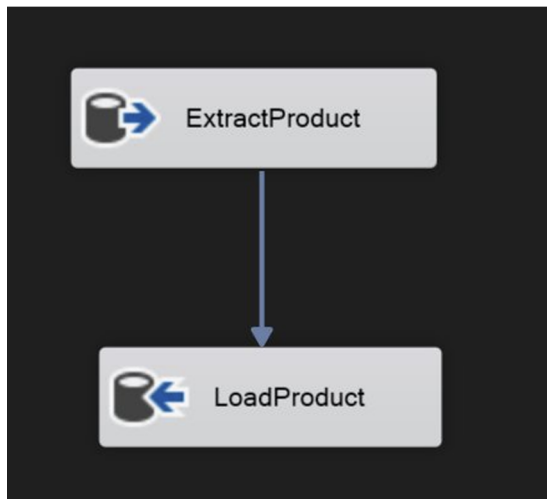
Le composant *Execute SQL Task* permet d'exécuter la même requête d'insertion dans le pipeline SSIS, offrant ainsi une traçabilité et une intégration dans un flux d'exécution contrôlé.

1.2.2 Data Flow — **LoadProduct**

Le Data Flow **LoadProduct** utilise un composant *Source OLE DB* (**ExtractProduct**) pour extraire les données produits via la requête suivante :

Listing 2 – Requête d'extraction des produits

```
1 SELECT P.ProductKey, P.EnglishProductName AS ProductName ,
2     P.Color, P.Size,
3     S.EnglishProductSubcategoryName AS ProductSubcategoryName ,
4     C.EnglishProductCategoryName AS ProductCategoryName
5 FROM AdventureWorksDW2017.dbo.DimProduct AS P
6 INNER JOIN AdventureWorksDW2017.dbo.DimProductSubcategory AS S
7     ON P.ProductSubcategoryKey = S.ProductSubcategoryKey
8 INNER JOIN AdventureWorksDW2017.dbo.DimProductCategory AS C
9     ON S.ProductCategoryKey = C.ProductCategoryKey;
```



(a) Data Flow — LoadProduct

SQLQuery1.sql - D:\...CLJO\LENOVO (77)* --vsBE2F.sql - DES...2CLJO\LENOVO (69)*

```

USE DWH
SELECT COUNT(*) FROM dbo.Products
  
```

100 %

Résultats Messages

(Aucun nom de colonne)

1	1588
---	------

(b) 1 588 produits chargés

Réponse

Quels sont les avantages du Data Flow par rapport au SQL Task ?

Le composant Data Flow offre plusieurs avantages :

- **Transformations visuelles** : possibilité d'ajouter des composants de transformation (tri, agrégation, lookup, nettoyage) entre la source et la destination
- **Support multi-sources** : compatible avec SQL Server, Oracle, Azure SQL, fichiers plats, etc.
- **Débogage avancé** : visualisation des données colonne par colonne avec les visionneuses de données
- **Gestion des erreurs** : redirection des lignes en erreur vers un flux séparé
- **Performance** : traitement en mémoire tampon pour les grands volumes de données

Le SQL Task, lui, est limité à l'exécution d'une requête SQL statique sans possibilité de transformation intermédiaire.

1.2.3 Data Flow — Load Date

Listing 3 – Requête d'extraction de la dimension Date

```

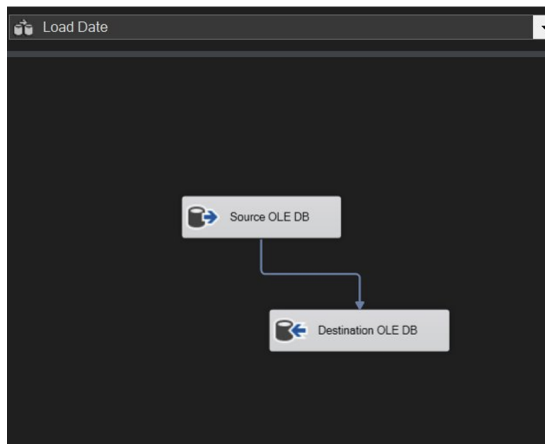
1 SELECT DateKey, FullDateAlternateKey AS FullDate,
2    SUBSTRING(CONVERT(CHAR(8), FullDateAlternateKey, 112), 5, 2)
3    + ' ' + EnglishMonthName AS MonthNumberName,
4    CalendarQuarter, CalendarYear
5 FROM AdventureWorksDW2017.dbo.DimDate;
  
```

1.2.4 Data Flow — LoadSales

Listing 4 – Requête d'extraction de la table de faits

```

1 SELECT C.CustomerDwKey, FIS.ProductKey, FIS.OrderDateKey,
2    FIS.OrderQuantity, FIS.SalesAmount,
  
```



(a) Data Flow — Load Date

SQLQuery1.sql - D:\CLJO\LENOVO (77)* X ~vsBE2F.sql - DES...2CLJO\LENOVO (69)*

```
USE DWH
SELECT COUNT(*) FROM dbo.Dates
```

100 %

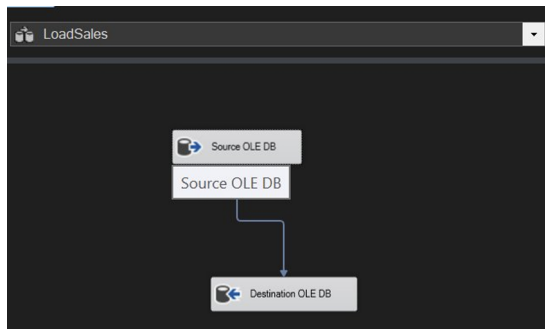
Résultats	
(Aucun nom de colonne)	
1	14608

(b) 14 608 dates chargées

```

3  FIS.UnitPrice, FIS.DiscountAmount
4  FROM AdventureWorksDW2017.dbo.FactInternetSales AS FIS
5  INNER JOIN AdventureWorksDW2017.dbo.DimCustomer AS Cust
6    ON FIS.CustomerKey = Cust.CustomerKey
7  INNER JOIN DWH.dbo.Customers AS C
8    ON (C.CustomerKey = Cust.CustomerAlternateKey
9        COLLATE French_CI_AS);

```



(a) Data Flow — LoadSales

SQLQuery1.sql - D:\CLJO\LENOVO (77)* X ~vsBE2F.sql - DES...2CLJO\LENOVO (69)*

```
USE DWH
SELECT COUNT(*) FROM dbo.FactSales
```

100 %

Résultats	
(Aucun nom de colonne)	
1	664378

(b) 664 378 ventes chargées

1.3 Ordre d'exécution et Sequence Container

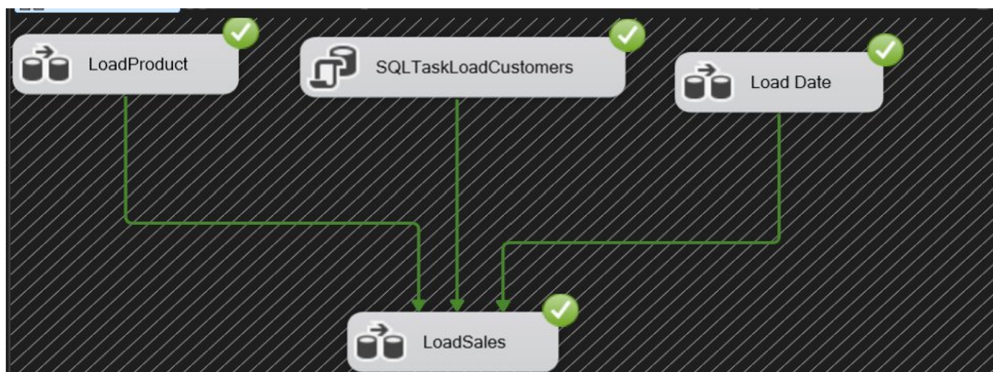


FIGURE 6 – Exécution réussie du package load_by_SSIS.dtsx — tous les composants en vert

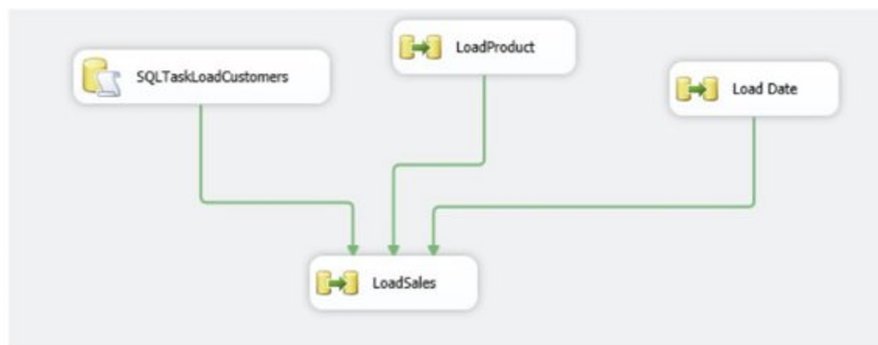


FIGURE 7 – Ordre d'exécution défini dans le lab (référence)

Réponse

Pourquoi l'ordre de chargement doit-il suivre cet ordre ?

L'ordre de chargement respecte les dépendances entre les tables :

- La table **FactSales** contient des clés étrangères vers les dimensions **Customers**, **Products** et **Dates**
- Si on chargeait les faits avant les dimensions, les références seraient invalides et les contraintes d'intégrité référentielle échoueraient
- Il faut donc absolument charger les **trois dimensions en premier** (en parallèle possible), puis charger la **table de faits en dernier**

Cela garantit que toutes les clés étrangères dans **FactSales** trouvent leur correspondance dans les dimensions.

1.4 Récapitulatif des données chargées

Table	Lignes chargées	Méthode
dbo.Customers	92 420	SSMS + SQL Task SSIS
dbo.Products	1 588	Data Flow SSIS
dbo.Dates	14 608	Data Flow SSIS
dbo.FactSales	664 378	Data Flow SSIS

TABLE 1 – Résumé du chargement des données dans DWH

2 Partie B — Slowly Changing Dimensions (SCD)

2.1 Contexte et préparation

Les *Slowly Changing Dimensions* (SCD) permettent de gérer l'évolution des données dans les dimensions d'un entrepôt de données. Nous avons ajouté deux colonnes de métadonnées à la table `Customers` :

Listing 5 – Ajout des colonnes de suivi temporel

```

1 ALTER TABLE DWH.dbo.Customers
2 ADD ValidFrom DATE, ValidTo DATE;
```

Un nouveau package `SCD_Customers.dtsx` a été créé. Les données source proviennent de la table `stg.CustomerInformation` dans la base `sample_dwh`.

2.2 Configuration du SCD Wizard

2.2.1 Mapping des colonnes et types SCD

Dimension columns	Change type
BirthDate	Fixed attribute
EmailAddress	Changing attribute
FullName	Changing attribute
Gender	Fixed attribute
MaritalStatus	Historical attribute
CountryRegion	Historical attribute

FIGURE 8 – Types SCD configurés pour chaque colonne de la dimension `Customers`

Colonne	Type SCD	Comportement
CustomerAlternateKey	Business Key	Clé métier (identifiant source)
BirthDate	Fixed attribute	Ne change jamais
Gender	Fixed attribute	Ne change jamais
EmailAddress	Changing (Type 1)	Écrase l'ancienne valeur
FullName	Changing (Type 1)	Écrase l'ancienne valeur
MaritalStatus	Historical (Type 2)	Crée une nouvelle ligne
CountryRegion	Historical (Type 2)	Crée une nouvelle ligne

TABLE 2 – Classification des attributs SCD pour la dimension Customers

FIGURE 9 – Configuration des dates ValidFrom et ValidTo dans le SCD Wizard

2.2.2 Options historiques

Les colonnes `ValidFrom` et `ValidTo` sont utilisées pour identifier les enregistrements courants et expirés. La variable système `System::StartTime` est utilisée pour horodater automatiquement les changements.

2.2.3 Data Flow SCD généré

Le wizard génère automatiquement :

- Une **Colonne dérivée** pour calculer les dates de validité
- Une **Commande OLE DB** pour les mises à jour Type 1
- Une **Destination d'insertion** pour les nouvelles lignes Type 2
- Un composant **Union All** pour fusionner les flux

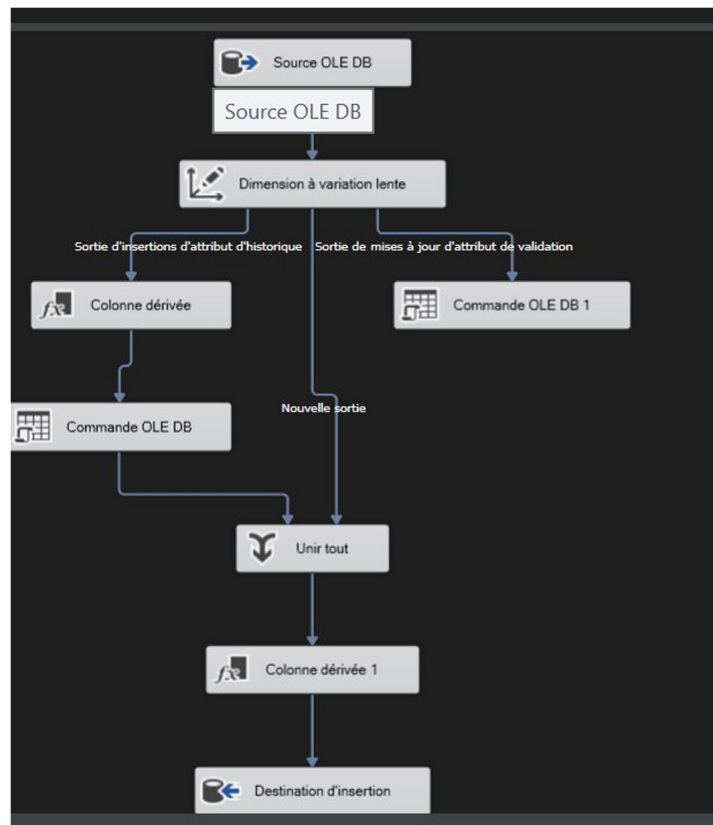


FIGURE 10 – Data Flow généré automatiquement par le SCD Wizard

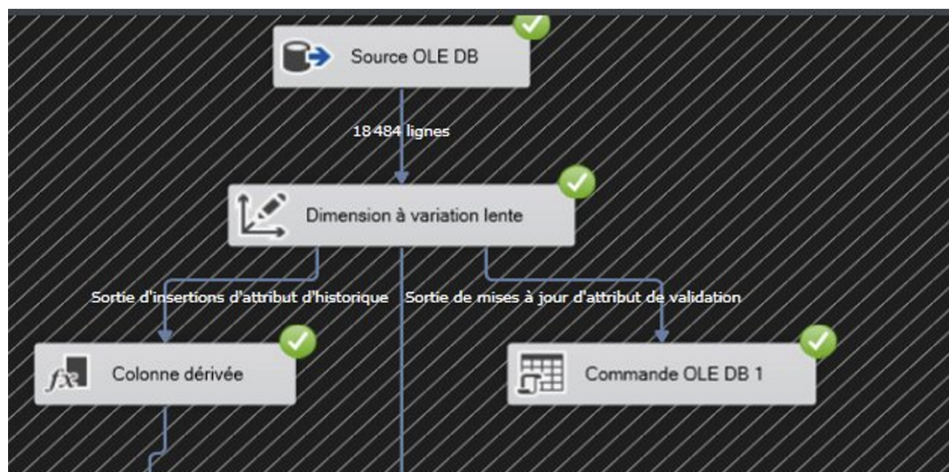


FIGURE 11 – Exécution réussie du package SCD_Customers

	CustomerAlternateKey	EnglishCountryRegionName	EmailAddress	MaritalStatus
1	AW00011000	France	test1@modified.com	S
2	AW00011001	Germany	test2@modified.com	M
3	AW00011002	Spain	test3@modified.com	S

FIGURE 12 – Résultats Type 2 — nouvelles lignes créées pour historiser les changements

2.3 Exécution et résultats SCD

Réponse

Différence entre l'insertion d'une nouvelle ligne et la mise à jour d'une ligne existante ?

Insertion (nouveau client) : Une nouvelle ligne est créée directement dans la table Customers avec ValidFrom = date_courante et ValidTo = NULL (enregistrement actif).

Mise à jour Type 2 (changement historisé) :

1. L'ancienne ligne est **fermée** : ValidTo = date_courante (elle devient expirée)
2. Une **nouvelle ligne** est créée avec les nouvelles valeurs et ValidFrom = date_courante

Cela permet de conserver l'historique complet des changements. On peut ainsi reconstituer l'état d'un client à n'importe quelle date passée.

Mise à jour Type 1 (écrasement) : La valeur est simplement mise à jour sur la ligne existante, sans historisation. L'ancienne valeur est perdue.

2.4 Historisation des suppressions

Cette partie traite d'un cas particulier : que se passe-t-il lorsqu'un client est supprimé du système transactionnel ? L'entrepôt doit conserver la trace de ce client mais le marquer comme inactif.

2.4.1 Architecture du Data Flow

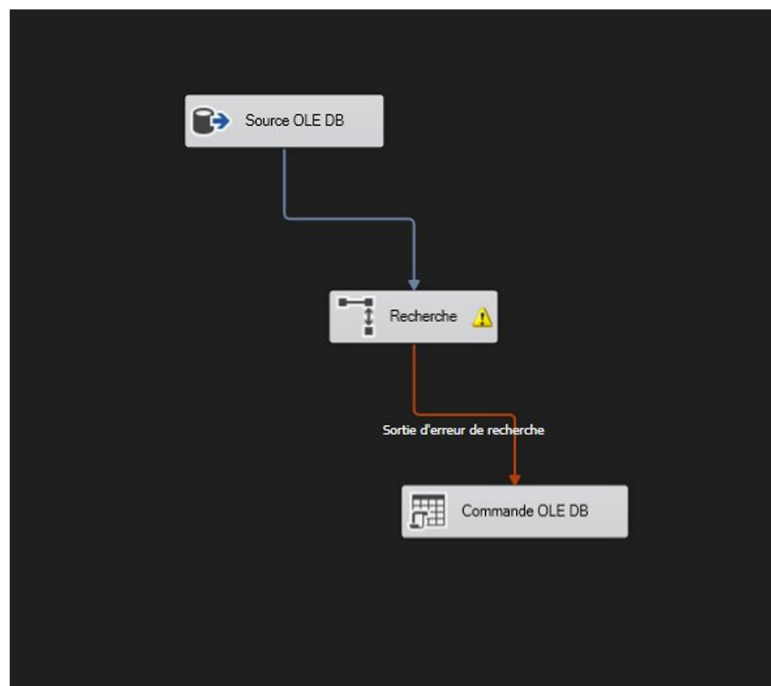


FIGURE 13 – Data Flow pour l'historisation des suppressions

Le flux se compose de trois étapes :

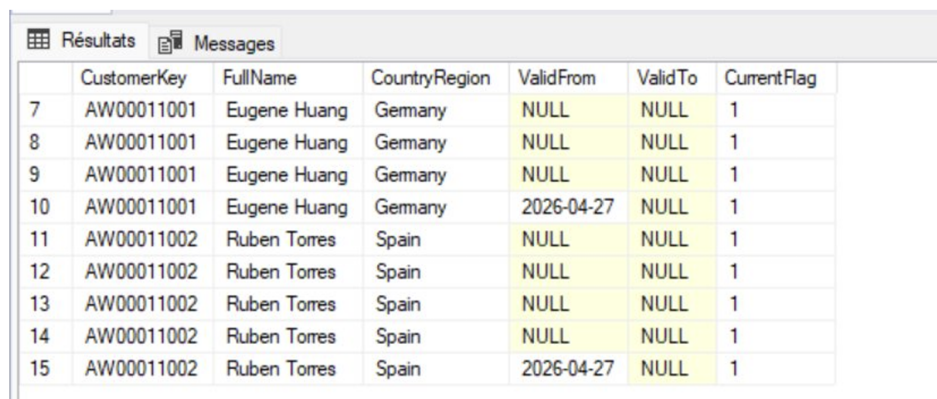
1. **Source OLE DB** : extrait les clients actifs (`CurrentFlag = 1`) depuis `DWH.dbo.Customers`
2. **Lookup** : recherche chaque client dans `stg.CustomerInformation`. Les clients non trouvés (supprimés) sont redirigés vers la sortie "no match"
3. **Commande OLE DB** : met à jour les clients supprimés

Listing 6 – Requête source — clients actifs du DWH

```
1 SELECT CustomerKey, FullName, EmailAddress, BirthDate,
2    MaritalStatus, Gender, Education, Occupation,
3    City, StateProvince, CountryRegion
4 FROM DWH.dbo.Customers WHERE CurrentFlag = 1
```

Listing 7 – Commande OLE DB — marquage des clients supprimés

```
1 UPDATE DWH.dbo.Customers
2 SET CurrentFlag = 0, ValidTo = GETDATE()
3 WHERE CustomerKey = ?
```



	CustomerKey	FullName	CountryRegion	ValidFrom	ValidTo	CurrentFlag
7	AW00011001	Eugene Huang	Germany	NULL	NULL	1
8	AW00011001	Eugene Huang	Germany	NULL	NULL	1
9	AW00011001	Eugene Huang	Germany	NULL	NULL	1
10	AW00011001	Eugene Huang	Germany	2026-04-27	NULL	1
11	AW00011002	Ruben Torres	Spain	NULL	NULL	1
12	AW00011002	Ruben Torres	Spain	NULL	NULL	1
13	AW00011002	Ruben Torres	Spain	NULL	NULL	1
14	AW00011002	Ruben Torres	Spain	NULL	NULL	1
15	AW00011002	Ruben Torres	Spain	2026-04-27	NULL	1

FIGURE 14 – Résultats après exécution — clients marqués inactifs (`CurrentFlag = 0`)

Réponse

Comment la table `Customers` a-t-elle été impactée ?

Les clients supprimés du système transactionnel (staging) ne sont **pas supprimés** de l'entrepôt de données. Ils sont simplement **marqués comme inactifs** :

- `CurrentFlag = 0` : indique que l'enregistrement n'est plus actif
- `ValidTo = GETDATE()` : date à laquelle le client a été désactivé

Cela permet de conserver l'historique complet tout en distinguant les clients actifs des clients inactifs.

Réponse

Que faut-il prendre en considération lors de l'utilisation de cette dimension pour le reporting ?

Pour les rapports portant sur les données **courantes** :

- Toujours filtrer avec `WHERE CurrentFlag = 1` pour n'inclure que les clients actifs
- Ou filtrer avec `WHERE ValidTo IS NULL` pour les enregistrements sans date de fin

Pour les rapports **historiques** :

- Utiliser `WHERE ValidFrom <= date_analyse AND (ValidTo >= date_analyse OR ValidTo IS NULL)` pour reconstituer l'état à une date précise
- Ne pas agréger sans filtrer, car un même client peut avoir plusieurs lignes (Type 2)

En résumé, la dimension **Customers** est une **dimension à historique lent** et tout rapport doit tenir compte du contexte temporel pour éviter les doublons ou les données obsolètes.

Conclusion

Ce laboratoire nous a permis de mettre en pratique plusieurs aspects fondamentaux de l'alimentation d'un entrepôt de données :

- **Chargement SSMS** : rapide et direct, mais sans traçabilité ni flexibilité
- **SQL Task SSIS** : intègre la requête dans un pipeline contrôlé
- **Data Flow SSIS** : approche la plus flexible, supportant les transformations, multi-sources et la gestion d'erreurs
- **SCD Type 1** : écrasement des valeurs changeantes sans historique
- **SCD Type 2** : historisation complète des changements via `ValidFrom/ValidTo`
- **Historisation des suppressions** : détection et marquage des enregistrements supprimés via `Lookup + OLE DB Command`

L'entrepôt DWH contient désormais plus de **664 000 faits de vente** reliés à **92 000 clients**, **1 500 produits** et **14 000 dates**, formant un schéma en étoile complet prêt pour l'analyse.